

Aqa: 纵向扩展 GPU 域中 LLM 的网络加速内存卸载

Abhishek Vijaya Kumar
康奈尔大学 美国伊萨卡
卡 abhishek@cs.cornell.edu

Gianni Antichi
米兰理工大学 意大利米兰
gianni@polimi.it

Rachee Singh
康奈尔大学 美国伊萨卡
rachee@cs.cornell.edu

抽象的

大语言模型 (LLM) 的推理受到 GPU 内存容量的限制。对云托管 LLM 的推理请求数量突然增加可能会耗尽 GPU 内存，从而导致多个提示之间对有限资源的争用。现代 LLM 服务引擎使用准入控制来应对有限 GPU 内存的挑战，这导致它们在请求突发期间没有响应。我们提出，抢先调度时间片中的提示对于确保响应式 LLM 推理至关重要，特别是在高负载和有限 GPU 内存的情况下。然而，抢占即时推理会产生较高的分页开销，从而降低推理吞吐量。我们提出了 Aqa，一个 GPU 内存管理框架，可以显着降低分页推理状态的开销；即使在突发请求模式下也能实现响应式和高吞吐量的推理。我们通过在具有 8 个 Nvidia H100 80G GPU 的服务器上托管几个最先进的不同模式的大型生成机器学习模型来评估 Aqa。与最先进的技术相比，Aqa 将 LLM 推理的响应速度提高了 20 $\times$ 。与单个长提示相比，它将 LLM 推理吞吐量提高了 4 $\times$ 。¹

CCS 概念: • 计算方法 → 机器学习;
• 网络 → 数据中心网络; 数据路径算法。

关键词: 大语言模型; 生成式人工智能; 寻呼; 虚拟内存; GPU 互连; 内存卸载;

ACM参考格式:
阿布舍克·维贾亚·库马尔、詹尼·安蒂奇和拉奇·辛格。2025.Aqa: 纵向扩展 GPU 域中 LLM 的网络加速内存卸载。第 30 届 ACM 国际编程语言和操作系统架构支持会议论文集, 第 2 卷 (ASPLOS '25), 2025 年 3 月 30 日至 4 月 3 日, 荷兰鹿特丹。ACM, 美国纽约州纽约市, 15 页。https://doi.org/10.1145/3676641.3715983

¹AQUA's code is available at <https://github.com/aquaml>.



This work is licensed under a Creative Commons Attribution International 4.0 License.

ASPLOS '25, March 30-April 3, 2025, Rotterdam, Netherlands

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1079-7/2025/03.

<https://doi.org/10.1145/3676641.3715983>

1 简介

近年来，用于从文本摘要到多媒体内容创建等多种任务的生成机器学习 (ML) 模型的普及度呈爆炸式增长[15,38,49,53]。这些模型通常托管在虚拟化多 GPU 云服务器上 [9、16、47、52]，这些服务器通过高带宽链路将服务器板上的一些 ML 加速器互连起来 [42]。

推理是内存密集型的。生成机器学习模型具有大量内存占用[5-7, 40]。例如，托管 Llama-405B 模型会消耗 800 GB 的 GPU 高带宽内存 (HBM)。此外，大语言模型 (LLM) 在使用注意机制处理输入提示时生成键值 (KV) 缓存[61]。KV 缓存随着输入提示的长度及其生成的序列呈二次方增长，每个提示占用几 GB 的 GPU 内存 [35]。因此，LLM 的推理请求数量突然增加可能会耗尽 GPU 内存，从而导致内存争用，因为多个提示会争夺 GPU 内存。

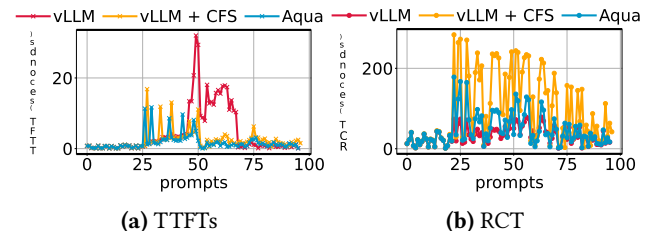


图 1. LLM 上推理查询的响应性（使用首次令牌时间或 TTFT 测量）和吞吐量（使用请求完成时间或 RCT 测量）。由于 vLLM 批处理查询，它具有低 RCT（高吞吐量）但高 TTFT（低响应性）。公平调度查询提高了响应能力，但分页开销在 RCT 中占主导地位。Aqa 通过高速多 GPU 互连（例如 Nvlink）卸载内存来减少分页开销，从而以低 RCT 实现响应式推理。

通过排队管理推理请求的突发。现代 LLM 服务引擎使用准入控制来应对有限 GPU 内存的挑战 [35, 54]。他们安排一批推理提示，并将剩余的提示排队，直到有可用的 GPU 内存。然而，这种带有准入控制的批处理范例很容易受到队头阻塞的影响[21]——在这种情况下，处理一批提示会延迟对队列中等待轮到的其他提示的响应。

排队会导致无响应。我们通过实验证明，排队提示会严重降低大语言模型推理能力。我们使用流行的服务引擎 vLLM [35] 在具有 80GB HBM 的 Nvidia A100 GPU 上提供 LLM。我们最初向 vLLM 发出 25 条提示，然后在 1 分钟内将请求率加倍。请求的激增耗尽了 vLLM 的 KV 缓存容量，导致后续提示排队。这会导致首次标记时间 (TTFT)（衡量模型开始响应速度的指标）激增 4 倍（图 1a 中的红线）。处理突发的另一种方法是跨额外的 GPU 自动缩放模型 [59]。但是，由于虚拟机启动延迟，横向扩展可能需要几分钟的时间，在此期间模型将无响应。由于即使是 3 秒的无响应都会导致用户放弃服务 [27]，因此数十秒的延迟会增加失去用户的风险。

我们的建议：可抢占推理。在这项工作中，我们假设时间片中提示的抢先调度对于实现响应式 LLM 推理至关重要，特别是在高负载和有限 GPU 内存的情况下。在此调度方案下，每个提示都将接收 GPU 计算的时间片，从而使模型保持响应能力。抢占式调度程序将在时间片边界停止当前提示的执行，并从 GPU 内存中调出其相关状态，为执行下一个提示腾出空间。使用时间片的抢占式调度还将打开在现有推理服务系统中实现细粒度公平调度的大门，这些系统仅通过准入控制支持粗粒度公平性[54]。

由于分页开销，抢占速度很慢。然而，抢占提示推理需要在时间片边界将提示状态从 GPU 内存中调出。如果提示的推理状态被分页到主机 DRAM [35, 55]，则将其传输回 GPU 内存会产生很高的开销。DRAM 和 GPU 内存之间传输状态的开销受外围组件互连 Express (PCIe) 连接带宽的限制。事实上，我们在 vLLM 中实现了一个抢占式公平调度程序，发现虽然抢占式调度改善了 vLLM 中的首次令牌时间（图 1a 中的橙色线），但它导致请求完成时间或完成推断提示的时间增加超过图 1b) 中的 2X (橙色线)。

Aqa：低开销的抢占式推理。在这项工作中，我们构建了 Aqa，一个 GPU 内存管理框架，它显著减少了 GPU 内存进出分页推理状态的开销，以实现高效的抢占式提示调度。减少的分页开销使 Aqa 能够两全其美——针对单个提示的响应式推理（图 1a），同时仍然实现高推理吞吐量（图 1b）。

阿卡的关键见解。ML 模型的推理可能会受到内存容量、内存带宽或计算的限制，具体取决于模型类型（例如，视觉与文本）和

推理查询负载 [55, 59, 69]。因此，在任何时候，云集群中的某些推理作业都未充分利用 GPU 内存，而其他推理作业则面临内存瓶颈。利用这一见解，Aqa 通过使 GPU 能够与需要更多内存的 GPU 共享备用内存，将内存分配与计算分配分离。Aqa 将这些内存卸载限制在云数据中心的纵向扩展域内的 GPU。纵向扩展域中的 GPU 通过高带宽 Nvlink 连接，使 Aqa 能够比 PCIe 上的传统主机 DRAM 更快地访问备用内存，从而显著减少抢占和分页开销。在现代数据中心中，多加速器服务器定义了一个扩展域。最近，纵向扩展领域已扩展到涵盖整个机架，例如 Nvidia Blackwell 的 72-GPU 架构，其中高带宽 Nvlink 互连所有 72 个 GPU。这些进步将 Aqa 解耦内存分配的优势扩展到整个机架 [41]。

技术贡献。Aqa 具有三个组件：Aqa-profiler 分析部署中托管 ML 模型的传入推理负载和 GPU 内存利用率的特征。使用此配置文件，Aqa-profiler 将模型标记为生产者或消费者，并将此信息提供给 Aqa-placer。生产商没有充分利用 GPU 内存，可以将多余的内存提供给其他模型。消费者面临 GPU 内存瓶颈，需要更多内存。Aqa-placer 放置模型，使生产者与同一 Nvlinks 域内的适当消费者配对。最后，Aqa-lib 是一个动态管理从消费者卸载到生产者 GPU 内存的张量的库。

具有低开销的抢占式公平调度。使用 Aqa 管理 GPU 之间的内存，我们构建了一个抢占式完全公平调度程序 [60] 以进行即时推理。我们设计了一种新算法来在不同推理阶段的提示之间划分批处理，以实现公平性，同时保留吞吐量优化[4]。由于 Aqa 支持对备用内存的快速访问，因此分页的开销很小。公平调度 (1) 实现了计算资源对所有提示的公平分配，(2) 允许模型即使在高负载和有限的 GPU 内存下也能保持响应。我们注意到，公平调度可以吸收临时的查询突发或管理持续的突发，直到自动缩放可以生成模型的新实例。

结果。我们通过在具有 8 个尖端 Nvidia H100 80G GPU 的服务器上托管多个不同模式（例如文本、音频、视觉）的最先进的大型生成 ML 模型来评估 Aqa。使用这些 ML 模型和推理工作负载的代表性分布，我们表明，与最先进的技术相比，Aqa 将 LLM 推理的响应能力（使用首次标记时间测量）提高了 20x。在内存受限的推理中，Aqa 将单个长提示的推理吞吐量提高了 4x [55, 69]。

2 背景和动机

机器学习推理任务现在对于许多组织来说至关重要[49]。最终用户通常在云服务器上托管机器学习模型，无需部署自己的硬件即可发出查询[16, 52]。生成模型可以生成各种媒体，包括文本[8]、图像[51]和音频[34]。LLM 与 ChatGPT 一样，因其文本生成功能而特别引人注目。

2.1 机器学习的基础设施

从本质上讲，机器学习推理需要执行多次大型矩阵乘法。因此，ML 推理使用 GPU 和 TPU 等专用加速器，针对快速矩阵乘法进行了优化[33]。除了 ML 加速器支持的浮点运算 (FLOP) 数量之外，由于 ML 模型大小的快速增长，加速器上的内存对于实现快速 ML 训练和推理也变得至关重要。生成式机器学习模型具有数万亿个参数，导致内存占用很大。为了满足模型的内存占用，GPU 配备了数十 GB 的高带宽内存或 HBM。通常，GPU 是使用 PCIe 连接到主机服务器的外围设备。主机具有用于通用计算的 CPU 和 DRAM[43–45]。

ML 的服务器目标。多加速器服务器（例如 Nvidia DGX [46]、Cerebras WSE [22]、Intel Gaudi [32]）已成为用于 ML 训练和推理的大型集群和数据中心部署的构建块。多加速器服务器由少量 ML 加速器组成，与高速板载电气互连（例如 Nvidia NVlink、Google ICI [33]）连接，支持加速器对之间高达每秒数百 GB 的带宽。云运营商使用网络结构将多加速器服务器机架连接到数据中心规模的部署中[33, 58]。

ML 集群中的带宽不对称。同一服务器上 GPU 之间的互连专门用于数百 GB/秒的高带宽[42]，而 GPU 和主机（例如 CPU、DRAM）之间的连接依赖于通用 PCIe（50GB/s），这比现代部署中的 Nvlink 慢得多。

2.2 LLM推理过程中的内存争用

LLM 通过将推理查询或提示转换为代表子词单元的标记来处理推理查询或提示。每个标记都被编码为捕获其含义的多维嵌入向量[24, 39]。在推理过程中，这些标记嵌入被输入到 LLM 中。给定用户提示，LLM 通过注意力机制[61]预测词汇表[20]中最可能的下一个标记，该机制通过将每个标记的嵌入与学习的键和值矩阵相乘来计算每个标记的键和值向量[24, 39]。一旦生成了最可能的标记，它就会被附加到序列中，其中包括提示和先前提出的标记，然后重复该过程。

该序列中所有标记的键和值向量（称为推理上下文）被缓存以进行有效的未来预测。同时处理的序列的键和值向量存储在 GPU 内存内的键值 (KV) 缓存中[35]。KV 缓存大小由序列长度和序列数量决定，随着生成令牌的数量而增长。在 GPU 上加载模型权重后，KV 缓存大小受到可用 GPU 内存的限制。当 KV 缓存超出可用 GPU 内存时，就会出现内存争用。

事实上，当模型对于 GPU 来说太大，或者提示的推理上下文 (KV 缓存) 和模型权重的组合内存超过 GPU 内存时，即使是单个提示也可能在推理过程中遇到饥饿。随着模型大小和支持的提示长度快速增长（例如 Llama-405B 和 Gemini 的 1M 提示），问题将会恶化。这促使最近努力将上下文从 GPU 分页到 DRAM，并仅在必要时加载它[55, 64, 69]。内存争用会阻碍缓存优化。最近，LLM 服务引擎正在缓存提示的长公共前缀的推理上下文[13, 25]。由于传入的提示会消耗内存，引擎必须丢弃 GPU 中缓存的上下文，从而使优化变得无用。相反，引擎唯一的选择是将推理上下文分页到 DRAM 或远程内存[37]。

2.3 缓解内存争用的现有方法

在 LLM 推理期间管理内存争用的一种方法是，当大量推理请求到达时，水平扩展托管模型的 GPU 数量[59]。然而，在更多 GPU 上生成模型实例会导致启动延迟几分钟[11, 59]。这些延迟限制了横向扩展方法对已接受并排队的请求的实用性，因为在配置更多资源之前它们将无响应。

寻呼面临带宽瓶颈。分页解决了内存争用引起的问题，但性能较差，因为 GPU 在等待分页数据时处于空闲状态。分页的根本瓶颈是 GPU 和 DRAM 之间的有限带宽，这受到 PCIe 带宽的限制。虽然更高的 PCIe 带宽可能会缓解这个问题，但研究表明 PCIe 带宽的增长速度一直很慢[26]。因此，当前将动态上下文卸载到主机 DRAM 的推理服务引擎必须在性能和响应能力之间进行权衡。事实上，为了证明这一点，我们在 vLLM 中实现了一个完全公平的调度程序，并发现虽然公平调度改善了 vLLM 中的首次令牌时间（图 1a 中的橙色线），但它导致请求完成时间或完成推断提示所需的时间增加 $\approx 50\%$ （图 1b 中的橙色线）。

3 阿卡设计

当推理负载增加时，由于 GPU 内存的原因，现代服务系统需要更长的时间来响应

容量争夺。图 1 显示，具有准入控制 [35, 54] 的批处理提示会对提示进行排队，直到当前批次的提示在 GPU 上完成执行，从而导致队头阻塞延迟。当 GPU 内存不足以支持模型或 KV 缓存和模型权重一起超过可用 GPU 内存时，即使推理负载只有一个提示，服务也可能无响应 [55]。Aqa 通过快速分页减少抢占开销并实现使用快速分页的抢占式调度程序，确保 GPU 内存容量争用下的响应式推理。

使用 Aqa 快速抢占。ML 集群中的多加速器服务器被分配给执行各种 ML 模型的推理作业。根据模型（例如图像与文本生成），推理作业具有不同的 GPU 计算核心、内存带宽和内存容量利用率。具有某些推理作业和工作负载的 GPU 受到具有大量备用内存容量的计算的瓶颈，而其他作业则受到内存容量的瓶颈 ([57], §4)。我们利用这种洞察力来解耦 GPU 上的内存和计算分配以进行推理作业。这使我们能够将 GPU 上的内存与空闲内存分配给另一个因不同 GPU 上的内存容量而出现瓶颈的作业。这使得内存受限的作业可以通过高速 GPU-GPU 互连（例如 Nvlinks）访问额外的内存，与 PCIe 相比，Nvlink 提供的带宽高出一个数量级。

Aqa 通过确保响应推理（首次令牌时间短）和保持高吞吐量（请求完成时间短）来解决云托管 LLM 中的推理请求突发问题。即使在突发情况下，Aqa 通过实现低寻呼开销的抢占式调度，也实现了响应灵敏和高吞吐量推理的目标 (§7)。Aqa 的快速抢占自然有利于依赖分页 [55, 64, 69] 的内存受限推理系统以及高级缓存优化 [13, 25]。在第 9 节中，我们展示了 Aqa 除了实现响应能力之外还提高了这些系统的性能。Aqa 具有三个主要组件（图 2）：

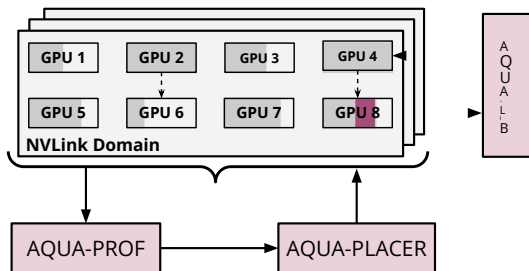


图 2. Aqa 的设计。

1. Aqa 分析器 (§4)。Aqa-profiler 可以分析任何托管模型的推理工作负载的特征以及相应的 GPU 内存利用率。基于

根据分析信息，Aqa-profiler 决定 GPU 是否有多余内存或需要更多 GPU 内存来维持其推理负载。我们将具有多余内存的 GPU 称为生产者，将需要内存的 GPU 称为消费者。

2. Aqa-placer (§5)。我们开发了一种算法 Aqa-placer，将模型放置在集群中，以最大限度地利用快速 GPU 间网络卸载内存的机会，而不是依赖附近偶然存在的空闲 GPU 内存。Aqa-placer 将模型放置在 GPU 上，以便将受内存限制的 ML 模型（即消费者）托管在内存丰富的 ML 模型（即生产者）附近。

3. Aqa-lib (§6)。推理工作负载可能会随着时间的推移而变化，这意味着推理过程中 GPU 内存的需求和供应都是有弹性的。这种动态环境使得将推理状态卸载到不同的 GPU 变得困难，而当 GPU 经历高负载的推理查询时，可能需要恢复内存。为了解决这个问题，我们开发了 Aqalib，这是一个内存管理框架，它公开了新的 Aqa 张量抽象。Aqa 张量是有弹性的——它们允许生产者 GPU 透明地回收它们暴露的内存，以便从内存消费者 GPU 卸载。

4 Aqa-profiler：分析内存消耗

Aqa-profiler 分析推理作业的当前 GPU 利用率及其在稳定状态下所服务的工作负载。使用此信息，Aqa-profiler 估计 GPU 是生产者还是消费者。我们将具有备用内存容量的 GPU 上的 ML 模型称为生产者，将受到内存容量瓶颈的作业称为消费者。

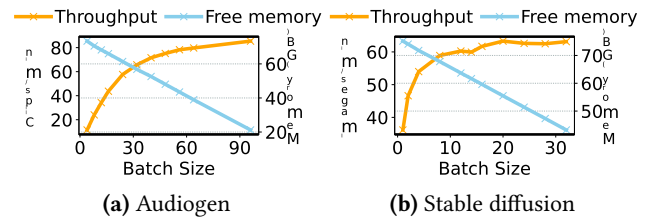


图 3. 该图显示，当音频和图像生成模型达到其吞吐量的稳定水平时，A100 80GB GPU 上有 10GB 的可用内存。事实上，将批量大小增加到超过某个点会导致吞吐量的增加递减。

我们的实验表明，不同的生成机器学习模型争夺不同的资源（例如，计算与内存）。具体来说，即使在峰值推理吞吐量下运行，视觉和音频生成模型也有充足的内存可供使用（图 3）。由于我们需要随时估计可用内存的大小，Aqa-profiler 通过增加批大小来测量可用内存，因为模型的内存消耗随着批大小的增加而增加。我们增加推理的批量大小，直到模型吞吐量饱和，以测量模型消耗的最高内存。由于图像和音频中的部分响应提供

由于效益有限，因此没有必要进一步增加批量大小。如果 GPU 上峰值吞吐量的可用内存高于阈值（例如 10 GB），我们将模型视为生产者。

4.1 大语言模型成为记忆的生产者吗？

我们增加推理批量大小直到吞吐量饱和的策略对于确定 LLM 的真实内存需求是无效的。这是因为 LLM 推理受益于部分响应，尤其是在交互式用例中。此外，大批量会对大语言模型响应时间服务级别协议（SLA）产生负面影响[4]。因此，我们开发了一种替代方法来对法学硕士进行分类。

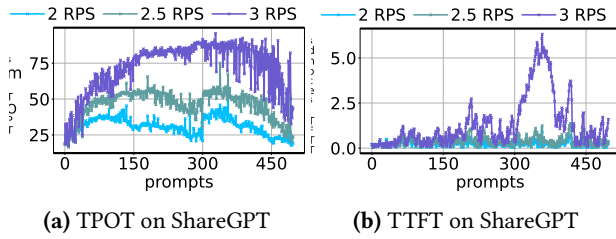


图 4.4a 显示了 80 GB A100 GPU 上 Llama 3.1 8B 模型的每个输出令牌的时间 (TPOT)。TPOT 随着请求率的增加而增加。图 4b 示出了到达第一个令牌的时间 (TTFT)，该时间随着传入请求的队列长度而上升。

LLM 的推理有两个主要阶段：预填充阶段和解码阶段。在预填充阶段，为提示中的每个标记计算键和值向量，并为每个标记与所有先前标记计算自注意力分数[61]。预填充阶段的输出是每个标记的向量，然后将其乘以矩阵以生成该层的输出，并保留下一层的输入维度。在解码阶段，每次前向传递都会生成新的令牌。与处理多个向量的预填充阶段不同，解码阶段每个提示仅处理一个向量。

最近的工作提出了推理调度程序，将某些输入提示的预填充阶段与其他提示的解码阶段进行批处理，以提高推理的整体效率[4]。最佳批量大小或块大小取决于托管 LLM 所需的 SLA。服务引擎对尽可能多的解码令牌进行批处理，并在必要时用新提示中的令牌补充批处理。

我们观察到，由于严格的响应性 SLA，托管 LLM 的 GPU 通常具有空闲内存，因为批处理受到块大小的限制。我们使用最近工作 [4] 中交互式用例的 SLA 要求来实证证明这一点，指定 100 毫秒的 P99 令牌之间时间 (TB T) 和 1 秒以下的第一个令牌时间 (TTFT)。我们还采用他们建议的 ShareGPT [35] 数据集块大小 512 来满足这些 SLA。

挑战。给定 SLA、块大小和数据集，Aqa-profiler 会搜索满足 SLA 的最高请求率，因为它会导致最大内存消耗。

横扫整个请求率空间是不可行且具有挑战性的。为了克服这个问题，我们首先在一个范围（例如 1-10 RPS）内执行二分搜索，然后以 0.5 RPS 增量和减量进行线性搜索以找到局部最大值。使用此功能，Aqa-profiler 以 LLM 在 SLA 内可以维持的最高流量速率输出可用内存。

在图 4 中，Llama 3.1 8B 型号可处理 2.5 RPS，且不违反 TTFT SLA。在 3 RPS 时，队列大小增加，导致 TTFT 为 5-6 秒（图 4b），这是不可接受的。因此，Aqa-profiler 将 2.5 RPS 的可用内存 (40 GB) 报告为该模型的可用内存，如果该值超过阈值（例如 10 GB），我们将该模型分类为生产者。

不可预测的负载。与批次中每个数据点的内存消耗相同的图像生成模型不同，LLM 推理批次中每个提示的内存消耗可能会根据输入和输出标记的长度而显著变化。Aqa-profiler 使用来自各个 LLM 部署的输入和输出令牌分布对分析请求进行采样 [17]。只要请求模式遵循历史分布，其可用内存估计就保持准确。即使进行了广泛的分析，GPU 的内存利用率也会随着不可预测的工作负载变化而波动。为了处理这个问题，Aqa 迅速回收内存并将其返回给生产者 (§ 6)。

剖析消费者。如果模型不是生产者，Aqa-profiler 会评估其对分页和抢占的需求。如果模型使用 Aqa-lib 分配了交换空间，则该模型被归类为消费者，并且 Aqa-profiler 确定所需的交换空间。对于内存受限推理和预填充缓存，该空间是根据提示数量、所需的最长序列以及推理上下文所需的内存来计算的。Aqa-profiler 通过不同大小的人工突发（例如每秒稳态请求的 2 倍、3 倍）凭经验确定 CFS 所需的交换空间（第 9.4 节）。如果不需要交换空间，我们将模型视为提供 0 内存的生产者。

极端情况下的性能。在极端情况下，当所有模型都是内存消耗者时，Aqa 默认使用 DRAM 进行分页，与 vLLM 和 FlexGen 等基准的性能相匹配，因为它们通过 PCIe 分页到 DRAM。

4.2 未来数据中心的 Aqa

机器学习基础设施的最新趋势表明，Aqa 的技术非常适合现代数据中心。Aqa 的目标环境是多个 ML 作业在高带宽互连域（例如 Nvlinks/NVSwitch 结构）中共存的环境。这些领域的规模正在不断扩大，例如 Nvidia 的 NVL72 机架（与 NVSwitch 互连的 72 个 GPU）[41] 和 Google 的 TPU 机架（每块 64 个 TPU）[33, 62]。这些扩展架构的大尺寸增加了不同机器学习工作负载在同一高带宽域中共存的可能性。多样化的工作负载

与较小的多 GPU 服务器相比, Aqa 能够匹配更多数量的生产者和消费者。

5 Aqa-placer: 最佳模型放置

Aqa-lib 在互连 GPU 上卸载张量的能力依赖于这些 GPU 上可用内存的可用性。Aqa-placer 将用于推理的 ML 模型映射到服务器集群中的 GPU, 目标是最大化在互连 GPU 上卸载 Aqa 张量的机会。将单个生产者映射到多个消费者是可行的, 但 Aqa-placer 在设计上不允许这样做, 并将一个生产者映射到一个消费者。与多个消费者共享生产者可能会超额订阅生产者 GPU 的 NVlink 带宽, 从而降低通过 GPU 间网络卸载内存的好处。

与稳定匹配的差异。将布局建模为简单的稳定匹配公式将假设任何生产者都可以与任何消费者匹配。然而, 在 Aqa 的背景下, 我们只能将生产者与通过相同的快速服务器规模 GPU 间网络 (例如 NVlinks 或 ICI) 连接的消费者配对。Aqa-placer 分两步解决模型分配问题。首先, 它将模型分配给多 GPU 服务器, 然后在每个服务器内, 它使用简单的稳定匹配将生产者与消费者进行匹配。我们将第一步编码为优化, 以生成服务器映射的最佳模型。

输入和输出。优化将集群中的服务器数量 S 、每个服务器的 GPU 数量 G 、模型数量 N 以及每个 GPU 的总高带宽内存容量 G_{mem} 作为输入。受当前一代多 GPU 服务器的启发, 我们做了一个简化的假设, 即服务器中的所有 GPU 都具有相同的硬件规格 (例如计算核心和内存) [43, 46]。关键输入是每个模型 R_m 的内存要求, 我们使用 Aqa-profiler 确定该要求。如果模型是生产者, 则模型的内存需求为正; 如果是消费者, 则模型的内存需求为负。例如, 图 3a 中的 Audiogen 将具有 $R_m = 20GB$ 。编码的输出是一个指示变量 $x_{m,s}$, 如果模型 m 映射到服务器 s , 则该变量为 1。为了处理分布式服务, 我们将指示变量 $x_{i,s}$ 分配给每个模型分片 i 。 P 是一个映射列表, 每个映射存储一组模型的所有张量并行分片。

将模型分配给 GPU 会受到以下限制: 每个 GPU 一个模型。每个模型必须映射到一台服务器:

$$\sum_{s=1}^S x_{m,s} = 1, \quad \forall m \in M \quad (1)$$

张量并行分片位于同一服务器上。张量并行 (TP) 分片通常放置在同一服务器上, 而管道并行分片则放置在不同服务器上 [4, 63]。因此, 我们将模型的所有 TP 分片映射到单个服务器。

$x_{p[j],s} == x_{p[j+1],s} \quad \forall j \in \{0 \dots \text{len}(p) - 1\}, \forall p \in P$ (2)
模型受每台服务器 GPU 数量的限制。在 Aqa 中, ML 模型分片恰好放置在一个 GPU 上。如果一个碎片

映射到单个 GPU, 映射到服务器的分片数量不应超过服务器上的 GPU 数量:

$$\sum_{m=1}^M x_{m,s} \leq G \quad \forall s \in \{1 \dots S\} \quad (3)$$

内存需求和供应平衡。我们将 mem_s 定义为映射到服务器的模型的内存需求 (R_m) 的总和。回想一下, R_m 可以是正数 (对于消费者) 或负数 (对于生产者)。因此, mem_s 在将模型映射到服务器后捕获服务器上可用的净内存。如果内存过剩, 则 mem_s 将为正值; 如果内存不足, 则 mem_s 将为负值。

$$mem_s = \sum_{m=1}^M x_{m,s} \cdot R_m \quad \forall s \in \{1 \dots S\} \quad (4)$$

消费者和生产者数量均衡。尽管使用等式 4 保持服务器上的内存使用平衡, 但编码可以找到将一个生产者和多个消费者映射到同一服务器的解决方案。我们计算映射到服务器的模型的净数量, 其中消费者计数为 -1 , 生产者计数为 $+1$ (在等式 5 中)。我们找到了 eq_s 接近 0 的解决方案, 以防止生产者带宽超额订阅。

$$eq_s = \sum_{m=1}^M x_{m,s} \cdot t \quad \forall s \in \{1 \dots S\}, \quad t = \frac{|R_m|}{R_m} \quad (5)$$

客观的。Aqa-placer 的目标是确保模型映射到服务器, 以便跨服务器不浪费 GPU 内存 (等式 4), 并且每台服务器拥有均衡数量的消费者和生产者 (等式 5)。我们通过选择服务器上内存利用率 (mem_s) 和模型数量 (eq_s) 的最大值并最小化它们的最大总和来实现此目的。为了使目标的两个部分具有相同的单位, 我们将 eq_s 与服务器的 GPU 内存相乘:

$$\max_s (mem_s) + G_s \cdot \max_s (eq_s) \quad (6)$$

我们使用商业优化求解器 Gurobi [31] 对算法进行编码, 发现对于最多 128 个 GPU 的集群, 它在 5 秒内达到最优解的 2% 以内。我们多久运行一次 Aqa-placer? 当生产者工作负载发生变化时, Aqa-lib 会快速为生产者回收内存, 从而限制消费者对快速分页的访问。虽然这可以防止生产者的性能下降, 但集群本身的状态可能不是最大化卸载机会的最佳状态。因此, 基础设施提供商必须在维持次优卸载或重新运行 Aqa-placer 之间进行权衡, 以重新映射作业与迁移, 以获得更好的共享机会。本文没有探讨这种权衡, 留待将来的工作。

6 Aqa-lib: 弹性内存管理

Aqa-lib 为 GPU 内存中的可迁移张量实现了一种新颖的抽象, 称为 Aqa 张量。Aqa Tensors 可以使用以下方式卸载到其他 GPU 的内存上

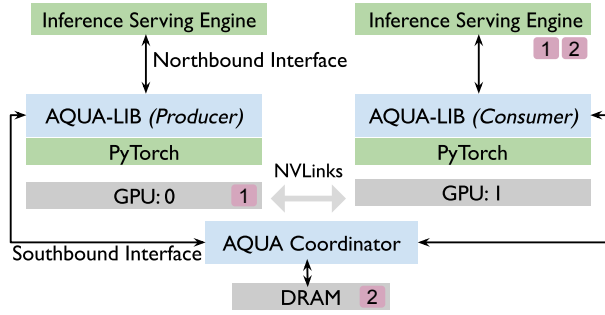


图 5. AQUA 的设计。GPU 1 正在托管消费者。Aqa-lib 知道 GPU 0 是生产者。Aqa-lib 允许 GPU 1 上的模型分配 Aqa 张量，这些张量被卸载到 GPU 0 的 HBM（显示在带有数字 1 的粉色框中）。如果 GPU 0 仅具有足够的内存来卸载一个张量，Aqa-lib 会回退到使用主机 DRAM（使用带有数字 2 的粉色框显示）。

快速的 GPU 间网络。在内部，Aqa-lib 根据服务器中所有其他 GPU 上的内存利用率和推理工作负载来决定何时应迁移 Aqa Tensors 以及迁移到哪个 GPU 内存。ML 模型开发人员不会注意到 Aqa 张量的物理位置的变化。我们将 Aqa 与两个流行的 ML 服务引擎 vLLM [35] 和 FlexGen [55] 集成，并且集成不需要对其代码库进行大的更改 (§8)。Aqa-lib 的接口。Aqa-lib 的实例在多 GPU 服务器的每个 GPU 上运行（图 5）。Aqa-lib 使用 Aqa-profiler 生成的配置文件进行初始化。Aqa-lib 使用此信息来决定 GPU 是否有空闲内存来充当生产者。Aqa-lib 实现了北向和南向接口。服务引擎使用北向接口与 Aqa-lib 交互。使用北向接口，服务系统还可以调频率与 Aqa-lib 共享其指标，例如推理负载（例如每秒请求数）和推理上下文大小。如果 GPU 是生产者并且服务引擎报告推理负载增加，Aqa-lib 将回收卸载的内存。当负载返回到 Aqa-profiler 分析期间确定的稳定状态值时，Aqa-lib 会将内存返回给消费者。当 Aqa 从生产者处提供内存或将其回收时，它使用北向接口通知推理服务引擎在事件发生后它有多少内存。南向接口使 Aqa-lib 能够与 Aqa 协调器维护的集中式线程安全数据存储进行通信。协调器跟踪来自消费者 GPU 的内存卸载请求和来自生产者 GPU 的内存提供。

中央协调员。中央协调员跟踪 HBM 的消费者和生产者。消费者 GPU 需要将张量卸载到任何潜在生产者 GPU 的 HBM 内存。协调器程序公开一组 REST 端点。Aqa-lib 使用这些 REST 端点来注册来自消费者 GPU 的内存请求并提供免费的内存

来自生产者 GPU 的内存。协调器还公开 REST 端点，允许生产者 GPU 在需要时回收内存。GPU 可能首先作为内存生产者，但如果其推理工作负载的特征发生变化，它可能会回收所有提供的内存。

分配 Aqa 张量。ML 模型导入 Aqa-lib 并使用 Aqa-profiler 生成的配置文件对其进行初始化。ML 模型使用 Aqa-lib 来实例化 Aqa 张量。Aqa-lib 使用南向接口通过适当的 REST 端点 (§8) 向协调器注册此分配请求。协调器跟踪可能已注册为生产者的 GPU。在模型开始在消费者 GPU 上执行之前，Aqa-placer (§5) 显式地选择哪个 GPU 将成为消费者 GPU 的生产者。协调器将卸载的张量位置的引用返回给消费者的 Aqa-lib。我们注意到，如果系统中不存在生产者 GPU，Aqa-lib 会回退到使用 DRAM 来卸载张量，就像之前的工作 [35, 55] 一样。释放分配的张量遵循类似的逻辑，消费者的 Aqa-lib 实例通知协调器，协调器集中管理 Aqa 张量的簿记。

回收 Aqa 张量。在生产者 GPU 上，Aqa-lib 轮询北向接口以查找 GPU 当前的内存利用率和工作负载统计信息。使用此功能，当生产者 GPU 上的推理负载增加时，Aqa-lib 允许生产者从卸载中回收内存。当 Aqa-lib 发现推理服务引擎报告的工作负载增加时，就会发生这种情况。Aqa-lib 使用 REST API 让协调器知道生产者 GPU 想要回内存。

Aqa-lib 控制循环。在消费级 GPU 上，Aqa-lib 的控制逻辑每隔几次推理迭代就会测试一次是否需要释放任何已卸载的 Aqa 张量。执行此检查需要 Aqa-lib 调用协调器的 REST API。此检查确保如果生产者注册了回收其内存的请求，消费者可以及时响应它。我们注意到，集中协调器的数据存储是线程安全的，可以防止多个 GPU 进行更改时出现意外行为。在我们的设计中，Aqa-lib 的开销非常低，因为与中央协调器的通信很少——每次可配置的推理迭代次数仅一次。我们在第 8 节中描述了 Aqa-lib 的进一步实现细节。

7 Aqa 的公平调度提示

当今推理服务系统中调度程序的默认行为是仅当有足够的 GPU 内存可用于其推理上下文时才接受新请求 [4, 35]。像 VTC [54] 这样的调度程序通过准入控制提供粗粒度的公平性，以避免高查询率的用户消耗所有计算周期。然而，当合法用户数量激增时，这种粗粒度的准入控制仍然会导致饥饿，

超出了分配的硬件的服务能力。如果 GPU 内存已被推断的提示完全占用，则任何其他请求都需要等待或挨饿，直到 GPU 内存中有足够的可用空间。即使使用自动缩放，提示也可能缺乏，因为配置新虚拟机会导致几分钟的延迟[11]。

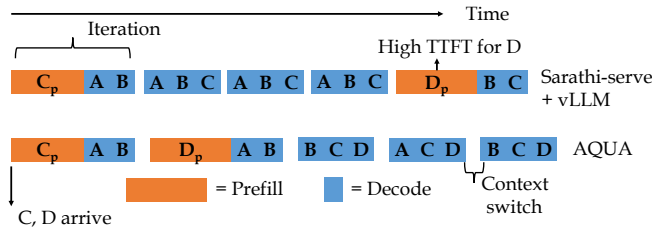


图 6. Aqa 在内存争用期间启用 CFS，此时 GPU 上一次只能容纳 5 个提示中的 3 个。

用于调度推理的批处理与分时。回想一下，最先进的调度算法通过对其他提示的解码计算来批处理预填充计算块，以提高计算效率 (§4)。该算法在批处理提示时优先考虑解码，然后当多个提示同时到达时，这会导致提示子集匮乏。在图 6 中，C、D 同时到达，但 D 一直处于饥饿状态，直到 A 完成执行，因为 GPU 的内存一次只能容纳 3 个提示。这与操作系统的行为形成鲜明对比，操作系统使用上下文切换来管理负载下的受限资源，并使用完全公平的调度程序 (CFS) [60]。上下文切换有两个步骤：抢占，将当前进程的状态（寄存器、页表等）保存到 DRAM；以及加载，将下一个进程的状态加载到 CPU 中。

挑战。天真地应用操作系统调度的时间切片，优先考虑接收最少计算时间的进程，破坏了最近的分块预填充优化[4]。这些优化利用了预填充受计算限制而解码受内存带宽限制的事实，允许预填充和解码令牌一起批处理，从而有效地提供“免费”解码。基于最少生成令牌数量的分时 GPU 只会批量预填充提示，因为它们生成的令牌为零，从而抵消了这一优势。

Aqa 的批量分区算法。在 Aqa 中，我们实现了公平调度，同时保留了分块预填充的优势。给定批量大小 b ，我们将其划分为 p 预填充令牌和 d 解码令牌。我们对分区的关键见解是将 d 设置为其上限，对应于 GPU 内存中适合的最大提示数。Aqa 首先用计算出的预填充标记数量最少的预填充提示填充 p ，用生成的标记数量最少的解码提示填充 d 。 d 中的剩余槽位分配给 p 中的提示。这确保了充分利用

当不存在预填充时，可以利用 GPU 容量，而当存在预填充时，可以利用分块预填充优化。如果 GPU 内存耗尽，Aqa 将停止填充批次。

为了简单起见，我们将时间片测量为推理迭代的数量，因为给定批量大小的每次迭代都花费相同的时间[66]。Aqa 在每 k 次迭代或请求完成时重新安排批次，调出属于下一批的提示，并调入不在 GPU 上的提示。图 6 显示了 Aqa 如何在预填充提示和解码提示之间公平分配计算，从而减少提示 D 的 TTFT。

高效的上下文切换。简单地使用 Aqa 张量作为交换空间并不是最优的，原因有两个。首先，vLLM 将与某一层关联的所有提示的键值张量存储为一个张量。因此，给定令牌的键和值张量分散在不连续的不同层的张量中，这导致卸载的 Aqa 张量有多个小副本。其次，Nvlinks 带宽对于小数据传输来说很差，例如，在两个 A100 GPU 之间，4MB 缓冲区的复制带宽为 50 GB/s，64 MB 缓冲区的复制带宽为 200 GB/s。我们通过将较小的张量收集到 GPU 上的临时张量并将其复制到卸载的张量来解决这一挑战。类似地，Aqa-lib 将卸载的数据复制到临时张量，并将其分散到各自较小的张量。

最后，服务引擎可以查询 Aqa-lib 的张量位置，并在张量位于 DRAM 上时从 CFS 回退到 FCFS，以避免高上下文切换开销。

其他调度策略。请注意，与简单地交换到 DRAM 相比，任何需要交换的策略都可以使用 Aqa 来实现，以获得更快的性能。例如，可以通过 Aqa 抢占优先级较低的 Aqa 张量托管的交换空间来实现优先级公平。只要有可能，Aqa-lib 就会将这些张量放在互连的 GPU 内存上。

8 实施

我们利用具有 500 行代码 (LOC) 的 PyTorch [1] 张量将 Aqa-lib 实现为可 pip 安装的 Python 库。该库提供了将标准的基于 CPU 的张量转换为 Aqa 张量的函数，并检索相应的 GPU 或 DRAM 指针以进行计算访问。在内部，Aqa-lib 与协调器进行通信。我们将库的北向接口与服务引擎集成，允许用户指定由 Aqa-profiler 管理的 GPU 内存提供和保留。

在 vLLM v0.5.3 中，我们实现了具有 150 个 LOC 的 CFS (§7)，以及 CUDA 内核，以有效地将小缓冲区收集到较大的缓冲区中，以便使用 200 LOC 进行 Nvlinks 传输。我们引入了一个具有唯一 ID 的新预填充缓存 API，并且 CFS 和预填充缓存共享相同的交换空间以实现上下文抢占。Aqa-lib 集成到 vLLM 和 FlexGen [55] 中，以在不到 50 个 LOC 的情况下使用 Aqa 张量而不是 PyTorch CPU 张量来管理交换空间。一把钥匙

Aqa Tensors 面临的挑战是将张量从 GPU 转移到 DRAM，而不导致数据损坏。我们通过以下见解解决了这个问题：推理引擎具有执行循环边界，无需 GPU 计算，从而允许安全传输。

我们使用 Gurobi [31] 在 700 LOC 中实现 Aqa-placer。Aqa 协调器是一个带有 REST API 的套接字服务器，分别作为生产者和消费者提供、回收和分配、释放内存，在 250 个 LOC 中实现。

9 评价

在本节中，我们将证明 Aqa 的快速抢占显著提高了响应能力和吞吐量。实验试验台。我们的实验利用两个独立的测试平台。主要测试台配备一台配备 8 个 NVIDIA H100 GPU 的服务器，每个 GPU 具有 80 GB 内存。这些 GPU 通过 Nvlinks 和 NVSwitches 互连，提供 450 GB/s 带宽，并通过 PCIe Gen5 连接到 DRAM，带宽为 50 GB/s，服务器具有 1.5 TB RAM。辅助测试床由一台带有两个 NVIDIA A100 GPU 的服务器组成，每个 GPU 具有 80 GB 内存，通过 Nvlinks 直接连接，带宽为 300 GB/s。这些 GPU 通过带宽为 25 GB/s 的 PCIe Gen4 连接到 DRAM。我们使用 H100 测试台进行端到端测试，使用 A100 服务器进行微基准测试，以最大限度地降低成本。

9.1 模型和工作负载。

数据集。我们使用 parti-prompts [28] 数据集来评估视觉生成模型，并使用 Audiogen 的默认提示来评估音频生成模型。根据最近的研究 [4]，我们使用 sharegpt [35] 和 arxiv summarization [14] 数据集来评估 LLM。

分布式服务。我们采用 vLLM 中的默认策略，即在推理期间使用张量并行性在 GPU 之间分配模型 [35, 56]，只要模型适合服务器并改变模型划分的 GPU 数量以表示不同的工作负载。具体来说，Llama 70B 可以使用张量并行性部署在 2 个 GPU 上，但当流量需求持续较高时，它也可以以更高的成本扩展到更多数量的 GPU。例如，当使用 H100 GPU 在 ShareGPT 数据集上运行时，从 2 个 GPU 增加到 4 个 GPU 使 Llama 70B 能够处理十倍以上的请求，因为 4 个 GPU 在加载模型权重后可提供充足的内存。但是，如果组织预计持续流量较低且偶尔会出现突发情况，那么使用较少的 GPU 并在出现突发情况时进行处理会更具成本效益。在我们的评估中，我们研究了这两种配置，并允许 Aqa-profiler 动态确定何时应充当生产者或消费者。

生产者工作负载。对于音频和视觉生成模型，我们使用首先达到峰值吞吐量的批量大小来执行模型。对于 Llama 8B 和 70B 模型，我们使用 ShareGPT 数据集并使用交互式工作负载 SLA [4] 运行它，并选择保持稳定队列长度的请求速率，即请求到达速率等于请求

完成率。我们在实验中使用与最近开发分块预填充 [4] 相同的块大小。

消费者工作负载。我们评估了三种不同的消费者工作负载：第 7 节中引入的需要公平调度（CFS）的突发工作负载、预填充缓存工作负载和内存受限的长提示工作负载。

突发期间的公平调度。我们使用两个模型来评估突发工作负载，1 个 GPU 上的 Yi-34B 和 2 个张量并行分布的 GPU 上的 Llama-70B。我们在 ShareGPT 数据集上确定两个模型的稳定请求率，并在 1 分钟的时间内将请求率加倍以创建突发。假配置新实例来处理突发的上限为 1 分钟，我们实现了最近的工作 [59] 中提出的最先进的方法来处理自动扩展时的负载平衡。我们每 5 秒跟踪一次请求到达率，每当请求率增加时，我们就会触发自动缩放通知。我们的工作负载生成器监听自动缩放通知，并通过首先等待一分钟让虚拟机启动来模拟负载均衡器的行为，然后将新到达的请求重定向到虚拟 http 服务器，直到突发中的请求数量被重定向。此时，所有实例都恢复以稳定的速率服务请求。我们使用虚拟服务器而不是虚拟机来避免分配额外的 GPU 并节省成本。

预填充缓存。我们使用 Yi-34B-200K 模型，在 2 个 GPU 上使用 vLLM，以拥有足够的 GPU 内存来容纳大提示并评估预填充缓存。我们选择一个包含 10 万个令牌的大型提示来缓存并将该提示与 ShareGPT 数据集一起交织。每次发出提示时，我们仅修改最后 256 个标记，以重用缓存的提示来建模用例，例如在长上下文中回答问题。

长提示。我们使用 FlexGen [55] 提供的 OPT-30B 模型来评估加载模型权重后超出 GPU 上可用内存的长提示。FlexGen 通过抢占上一层的上下文并加载下一层的上下文，将提示的推理上下文卸载到 DRAM。我们确定 arxiv 汇总数据集中的提示长度中位数超过 7000 个标记，因此我们对长度为 8192 的提示进行采样来评估 FlexGen，因为其评估脚本在实验期间需要单个提示长度参数。

9.2 基线

我们使用两个基准评估 CFS 工作负载：默认 vLLM 调度程序（FCFS）和带有 CFS 的 vLLM。虽然像 VTC [54] 这样的非抢占式准入控制调度程序可以防止恶意用户垄断计算资源，但它们仍然会在合法用户激增期间遭受排队延迟的困扰。为了模拟这种行为，我们的 vLLM 基线将突发中的每个请求视为新的唯一用户，捕获无响应情况。因此，我们图表中的 vLLM 线也代表 VTC 性能。

Model	Workload	Engine	GPUs	Type
Llama3.1 70B	ShareGPT	vLLM + CFS	2	C
Yi-34B-200K	ShareGPT + Prefill Cache	vLLM	2	C
OPT-30B	Long-prompt	FlexGen	1	C
Llama3.1 8B	ShareGPT	vLLM	1	P
Llama3.1 70B	ShareGPT	vLLM	4	P
SD-3, SD-XL, Kandinsky	Parti prompts	Diffusers	1	P
MusicGen, AudioGen	Audio descriptions	PyTorch	1	P

表 1. 模型、工作负载、服务引擎及其类型 (P = 生产者, C = 消费者)。SD = 稳定扩散。

我们使用带有预填充缓存的 vLLM 实现, 抢占 DRA M 作为预填充缓存的基准。我们使用普通 FlexGen 抢占 DRAM 作为评估长提示工作负载的基准。除非专门评估 CFS, 否则所有实验都使用默认调度程序。评估指标。我们使用第一个令牌的时间 (TTFT)、每个输出令牌的时间 (TPOT) 以及每秒令牌的解码吞吐量或生成吞吐量。

9.3 端到端评估。

我们在 8 个 GPU H100 服务器上运行端到端评估, 并将集群大小设置为 16 以避免高昂的成本。

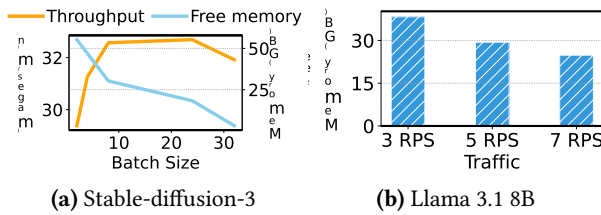


图 7. 图 7a 显示了 Aqa 在 Stable-diffusion-3 上生成的配置文件, 该配置文件在 8 个图像的批量大小左右达到峰值吞吐量, GPU 上有接近 30 GB 的未使用内存。图 7b 显示, 在符合 SLA 的 5 RPS 稳定速率下, Llama 8B 上有超过 25 GB 的可用内存。

模型是如何分类的? 图 7a 显示了 Aqa-profiler 在 H100 GPU 上为 Stable-diffusion-3 模型生成的跟踪。分析器在推理过程中增加批量大小, 直到吞吐量饱和, 记录相应的可用 GPU 内存。图 7b 显示了 Llama 3.1 8B 模型的轨迹。根据之前的工作 [4, 35], 我们选择 TTFT 低于 1 秒且令牌间延迟 (ITL) 低于 100 毫秒的 SLA, 针对批量大小为 512 的交互式工作负载。基于此 SLA 和 ShareGPT 输入请求分布, 我们的分析器测试各种请求速率并捕获服务期间的可用空闲 GPU 内存。我们的实验发现, SLA 内 LLM 的最大可持续流量为每秒 5 个请求 (5 RPS), 如图 7 所示。

类似地分析我们实验设置中的所有其他模型, 以对生产者和消费者进行分类。

将模型放置在 GPU 上。分析后, 我们从音频模型中采样 10%, 从视觉模型中采样 15%, 其余部分从表 1 中的模型中的 LLM (代表 LLM 的较高受欢迎程度) 中采样, 直到耗尽可用的 GPU。我们使用 Aqa-profiler 中的采样模型及其内存配置文件执行 Aqa-placer。

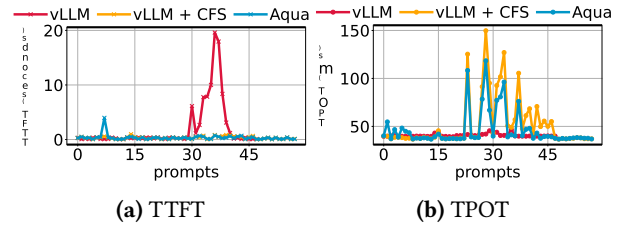


图 8. 图 8a 显示了到达使用张量并行性在 2 个 GPU 上运行的 Llama 3.1 70B 的请求的 TTFT。这些请求是从 ShareGPT 中采样的。图 8b 按顺序显示了请求的 TPOT。与 vLLM + CFS 相比, Aqua 最大限度地减少了 TTFT, 同时 TPOT 降低了 1.5x。

CFS 从 Aqua 中受益多少? Aqua-placer 将在 2 个 GPU 上运行 Llama 3.1 70B 的分布式 CFS 工作负载与两个单 GPU 模型 (Llama 8B 和 Audiogen) 相匹配。CFS 工作负载的两个分片使用 Aqa-lib 分配交换空间, 透明地放置在运行 Llama 8B 和 Audiogen 的 GPU 上。我们以稳定的 CFS 请求率启动工作负载, 并以此速率发出 25 次提示后, 工作负载生成器将请求率在一分钟内翻倍。这种激增由基线 FCFS 调度程序处理, 导致 TTFT 达到 20 秒, 如图 8a 所示。

具有 vLLM 的 CFS 策略可减少 TTFT, 但会因 DRAM 的抢占开销而产生较高的 TPOT (图 8b)。相比之下, 与 vLLM + CFS 相比, Aqua 不仅将 TTFT 减少了 20x, 还将 TPOT 减少了多达 1.5x。Aqua 还缩短了突发影响: 在图 8a 中, 使用 vLLM + CFS 时, TPOT 在 25-48 个请求之间保持较高水平, 但使用 Aqua 时仅在 25-40 个请求之间。这一改进源于 Aqa-lib 的快速抢占, 它通过 Nvlink 将数据透明地卸载到相邻的 GPU。图 8 还显示, 在前 25 个请求期间和第 50 个请求之后, 没有内存争用时, CFS 的行为与 FCFS 相同。

预填充缓存的改进。Aqa-placer 将在 2 个 GPU 上运行的分布式 Yi-34B-200K 与两个单 GPU 制作者 (Stable-Diffusion-3 (SD-3) 和 Kandinsky (Kand)) 相匹配。我们从稳定的请求率开始, 发出 100K 令牌提示, 使用 API 缓存除最后 256 个令牌之外的所有令牌的推理上下文。然后, 10% 的请求使用缓存的令牌作为具有不同 256 个令牌后缀的前缀, 而其余请求则从 ShareGPT 中采样。服务引擎最初计算上下文, 大约需要 10 秒, 将其缓存在交换空间中, 并将其加载到

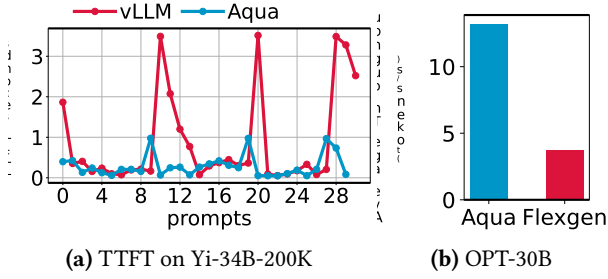


图 9. 图 9a 显示了按顺序发送到在 2 个具有张量并行性的 GPU 上服务的 Yi-34B-200K 的请求的 TTFT，其中由于从交换空间加载缓存的上下文而出现峰值。图 9b 显示了在 1 个 GPU 上服务的 OPT-30B 的解码吞吐量。Aqua 显著提高了两种工作负载的性能。

当具有相同前缀的请求到达时 GPU。图 9a 跟踪了缓存后的 30 个稳态请求。

基线显示每次缓存命中时 TTFT 峰值高达 4 秒，后续请求也面临高 TTFT，因为它们使用长提示进行批处理并等待将其上下文加载到 GPU 中。Aqua-lib 最大限度地减少了从交换空间到 GPU 的上下文加载时间，将长提示的 TTFT 减少了 3.5 $\times$ 。这也会降低使用缓存提示批处理的后续请求的 TTFT。

内存约束推理的改进。Aqua-placer 与在单个 GPU 上运行的 OPT-30B 与 Llama 3.1 8B 相匹配。我们使用 FlexGen 运行 OPT-30B，用于资源较少的离线推理场景 [48]，旨在通过有效地将上下文交换到 DRAM 来提高系统的吞吐量。我们测量了 FlexGen 在长度为 8192 的提示序列上持续 10 分钟的生成吞吐量，并报告了提示之间的平均吞吐量。加载模型权重后，带有 8K token 的提示生成的上下文太大，无法容纳 GPU，系统开始交换以执行推理。图 9b 表明，在整个实验过程中，与交换到 DRAM 的基线相比，Aqua 在生成吞吐量方面始终实现了 4 $\times$ 的提高。

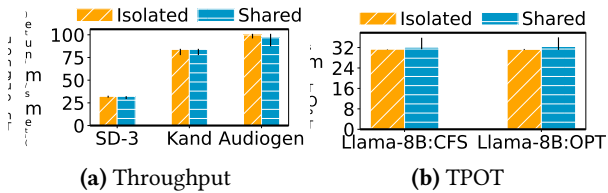


图 10. 图 10a 显示，当音频和图像制作者与消费者工作负载共享内存时，对吞吐量的影响可以忽略不计。孤立的条形图反映了没有消费者的生产者的表现。图 10b 显示内存共享使 2 个 Llama 8B 实例的平均 TPOT 增加了约 4.5 毫秒。

共享内存对生产者有什么影响？我们执行了所有共享内存的生产者

之前没有消费者的实验将其与与消费者共享内存时的性能进行比较。图 10 显示共享内存对所有模式的生产者的影响都很低。Audiogen 的吞吐量差异小于 5%，而视觉生成模型的吞吐量差异不存在。我们使用平均 TPOT 来衡量 LLM 生成器的性能，因为它捕获了执行一次推理迭代所需的时间。图 10b 显示共享内存对 TPOT 的影响约为 4.5 毫秒。正如最近的工作 [2] 所示，消费者访问生产者的 GPU 内存可能会导致内存带宽争用。我们认为 LLM 和 audiogen 模型性能下降的原因是它们依赖于已知会受到 GPU 内存带宽瓶颈的转换器层 [4]，任何带宽争用都会降低性能。

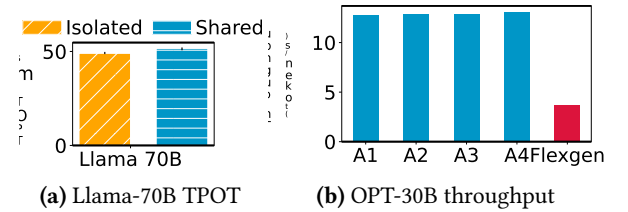


图 11. 图 11a 显示，在 ShareGPT 数据集上使用张量并行性，在 4 个 GPU 上提供服务的 Llama 70B 上，共享内存对 10 分钟内平均 TPOT 的影响为 5%。图 11b 显示了相同持续时间内 4 个 OPT-30B 实例的吞吐量，始终比基线好 4 $\times$ 。（阿=阿卡）

对分布式生产者有什么影响？由于 Aqua-placer 没有将任何消费者与分布式 LLM 生产者相匹配，因此我们在本实验中研究了通过 Nvlink 共享内存对分布式 LLM 性能的影响。张量并行推理需要通过 Nvlink 进行 GPU 间通信，当消费者访问生产者 GPU 上的内存时也会使用 Nvlink，这可能会影响性能。为了评估这一点，我们在 4 个具有张量并行性的 GPU 上运行 Llama 3.1 70B 模型，并在另外 4 个 GPU 上运行 4 个 OPT-30B 实例。为 Llama 服务的每个 GPU 都有 15 GB 的可用空间，并且是 OPT-30B 之一的生产者，该 OPT-30B 需要 10 GB 交换空间来推断 8K 令牌提示。我们选择 OPT-30B 进行此实验，因为它经常利用 Nvlink 来交换推理上下文。

我们运行了 10 分钟的实验，其中 Llama 处理稳定的请求，OPT-30B 根据长提示生成令牌。图 11 显示共享内存导致 Llama 上的 TPOT 开销可以忽略不计，为 5%。我们调查了低开销的原因，并确定了两个原因：首先，张量并行期间的通信大小只有几兆字节。其次，这种通信发生在层之间，使得互连在计算期间处于空闲状态。小缓冲区大小与以下事实相结合

计算期间互连处于空闲状态，导致 Nvlink 利用率不足。因此，消费者的流量几乎不会产生任何额外的影响。我们相信，如果 NVIDIA 公开一个 API 来设置数据包的优先级，我们可以使用该 API 将张量并行数据包优先于生产者-消费者流量，则可以消除这种开销。

9.4 Aqa 的适应性

在本小节中，我们使用 A100 测试台演示 Aqa 对不同生产者工作负载需求的适应性以及 Aqa 对不同数据集的可扩展性。

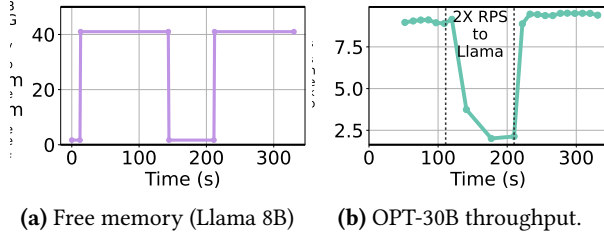


图 12. Aqa 的弹性机会性地将内存受限工作负载的吞吐量提高了 6 $\times$ 。

阿卡的弹性。在本实验中，我们在具有 2 个 A100 GPU 的 GPU 服务器上使用 Aqa 托管 Llama 3.1 8B 作为生产者，将 OPT-30B 作为消费者作业。我们改变发送给生产者的推理请求的数量。图 12a 显示了为 Llama 8B 提供服务的 GPU 上的可用空闲内存。开始时，服务引擎保留所有内存来容纳传入请求的推理上下文。Aqa-lib 注意到推理负载稳定，并使用分析数据向消费者提供该 GPU 上的内存。

在图 12a 中的 60 秒标记附近，我们开始长提示推理，实现每秒 8 个令牌的高吞吐量。在 120 秒处，我们将生产者的流量增加了一倍。这会增加待处理队列中的请求数量，Aqa-lib 会注意到这一点并快速回收内存以适应负载。回收内存会降低消费者的吞吐量（图 12b）。当 LLM 完成所有请求的服务后，吞吐量再次增加。Aqa-lib 再次检测到稳定的流量，并透明地将消费者卸载的张量移回生产者的 GPU。这表明 Aqa 可以动态响应生产者工作负载的变化，并伺机提高消费者性能。

Aqa 的优势扩展到不同的数据集。在本实验中，我们使用 arxiv 总结数据集和 Yi-34B 来展示 Aqa 的优势如何延续到其他数据集。arxiv 数据集的提示较长，提示中位数约为 7000 个令牌，而 ShareGPT 数据集中有 2000 个令牌。因此，我们首先使用 Aqa-profiler 来分析在单个 A100 GPU 上执行的 Yi-34B 处理突发的交换空间需求。给定稳定的请求率，分析器会创建稳定请求率的倍数的突发

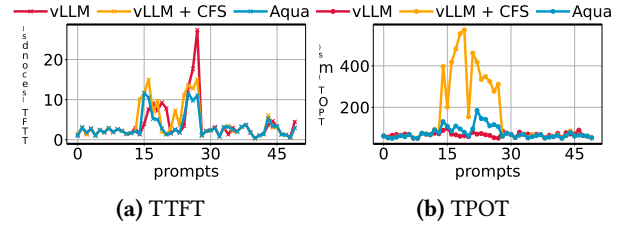


图 13. 图 13a 显示了到达在 1 个 A100 GPU 上运行的 Yi-34B 的请求的 TTFT。请求是从 Arxiv 数据集采样的。图 13b 显示了请求的 TPOT。

请求率，并确定模型需要 6GB、15GB 和 40GB 交换空间来处理一分钟的突发，这是稳定请求率的 2 $\times$ 、3 $\times$ 和 5 $\times$ 。

我们首先在一个 GPU 上托管 Llama 8B，在另一个 GPU 上托管 Yi-34B，以稳定的速率发出请求。为了模拟突发，我们将 Yi-34B 的请求率提高三倍，持续一分钟。图 13 显示 Aqa 有效地处理突发，通过快速抢占将峰值 TTFT 降低 2 $\times$ ，并将 TPOT 降低 2 $\times$ 。所有系统中 10-20 之间提示的 TTFT 都较高，因为提示很长，并且预填充阶段主要受到计算瓶颈。

10 相关工作

GPU 作业调度程序。Usher [57] 是最新的调度程序，它分析 GPU 上的计算和内存利用率，并在 GPU 上复用互补模型以提高资源利用率。Alpa-serve [36] 通过将其他工作负载中未充分利用的 GPU 分配给具有高流量的作业，动态扩展分配给作业的 GPU。类似地，Salus [68]、clockwork [30]、Orion [18] 和 PipeSwitch [19] 在模型、层和内核粒度上复用模型，以有效地分时共享 GPU 上的计算。所有这些系统在复用模型时都将计算和 GPU 上的内存分配紧密耦合，并且在计算饱和时错过了利用可用内存容量的机会。Aqa 弥补了这一差距，即使在没有机会共享计算的情况下，也能最大限度地提高多 GPU 服务器上的内存利用率。

在线 LLM 推理的调度程序。最近，社区提出了许多用于安排 LLM 提示的算法 [4,54,59,65]。Sarathi-serve [4] 提出了一个调度程序来批量预填充和解码。Llumnix [59] 通过负载平衡、自动缩放和实时迁移提示来跨 LLM 实例安排提示。VTC [54] 通过准入控制对 LLM 服务引擎进行粗粒度公平调度，以防止不良行为者压垮基础设施。VTC 还提到，抢占式调度与他们的工作正交，应该在将来解决。Aqa 中的 CFS 通过在自动缩放启动时处理突发以及公平地将资源分配给承认的提示来补充这些系统。MLFQ 等努力 [65]

还构建抢占式调度程序，Aqa-lib 可以通过快速抢占进一步提高其性能。

CUDA 统一虚拟内存。CUDA 统一内存提供跨越 GPU 和主机内存的单一虚拟地址空间。CUDA 驱动程序管理页面错误和预取，允许 GPU 在所有 GPU 和 DRAM 之间无缝分配和访问内存。然而，它的默认预取策略对于 LLM 推理来说是不够的，其中推理上下文分散在不连续的内存区域中。此外，CUDA 驱动程序不会机会性地利用 NVLink 和 NVSwitch 互连上的空闲 GPU 内存。之前的工作，例如 Memory Harvester [23]，通过利用空闲 GPU 内存作为驱动程序级别的交换空间来解决这一限制，但仍然与应用程序无关，使工作负载无法控制逐出和预取。缺乏控制可能会导致分页效率低下，例如逐出模型权重而不是推理上下文，从而增加开销。相比之下，LLM 服务引擎在应用程序层手动管理预取和逐出，以优化内存使用。Aqa 通过实现对预取顺序的显式控制并伺机将卸载的数据迁移到更快的 GPU 内存，从而最大限度地提高 LLM 推理的分页性能，从而改进了 CUDA 统一内存。

其他生成引擎。Orca [67] 是一个 LLM 推理引擎，引入了连续批处理，而 vLLM [35] 通过设计分页注意力改进了这一点。Deepspeed-zero [12] 是 FlexGen [55] 的基线。鉴于 Aqa 可以增强 FlexGen 的性能，类似的优势也可以扩展到 Deepspeed。HeteGen [69] 基于 FlexGen，利用 CPU 和 GPU 进行计算。由于 Het-eGen 也依赖 PCIe 进行 CPU 到 GPU I/O，因此 Aqa 是对其的补充，可以进一步提高其性能。与 CacheGen [37] 以压缩精度为代价并将许多提示存储在远程内存中不同，Aqa 缓存上下文的同时保持精度，适合使用医疗保健、法律和金融等关键数据的场景。我们注意到有模拟器 [3] 有助于快速分析。Aqa-profiler 还可以与此类模拟器交互并更快地进行分析，但我们在这项工作中直接从硬件进行测量以确保准确性。Dist-serve [70] 和 Splitwise [50] 建议在预填充和解码阶段使用不同的硬件。由于 Aqa-profiler 可以将分类工作负载分类为生产者或消费者，Aqa 是对它们的补充。

在 RDMA 上交换 DRAM 页面。Infiniswap [29] 和 Fastswap [10] 通过 RDMA 收集内存并在互连服务器的 DRAM 上移动页面，从而加速操作系统中的分页。这些系统中的内存分配算法不能保证带宽隔离，因为同一个生产者可以在多个消费者之间共享。了解 Nvlinks 和 NVSwitch 等封闭系统在争用情况下的行为对于

提供寻呼带宽保证，但由于其拥塞控制算法是闭源的，因此具有挑战性。尽管 Aqa 的设计允许我们实现类似的内存分配算法 [10, 29] 来跨 GPU 分配内存，但我们还是开发了 Aqa-placer，因为我们希望在分页期间提供带宽保证。我们将在争用期间对 Nvlink 上的寻呼带宽进行建模的探索留给未来的工作。

11 致谢

这项工作得到了 ACE 的部分支持，ACE 是 JUMP 2.0 的七个中心之一，JUMP 2.0 是 DARPA 赞助的半导体研究公司 (SRC) 项目。这项工作的作者还得到了 NSF 奖 # 2444537 和 Cornell Bowers CIS-Linkedin Grant 的支持。

参考

- [1] 2024. PyTorch: 一种开源机器学习框架，可加速从研究原型到生产部署的过程。https://pytorch.org。访问时间：2024 年 5 月 3 日。
- [2] 萨克沙姆·阿加瓦尔 (Saksham Agarwal)、阿尔温德·克里希那穆蒂 (Arvind Krishnamurthy) 和拉奇特·阿加瓦尔 (Rachit Agarwal)。2023. 主机拥塞控制。ACM SIGCOMM 2023 会议记录 (<conf-loc>、<city>纽约</city>、<state>NY</state>、<country>USA</country>、</conf-loc>) (ACM SIGCOMM '23)。计算机协会，美国纽约州纽约市，275–287。https://doi.org/10.1145/3603269.3604878
- [3] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav Gulavani, Ramachandran Ramjee 和 Alexey Tumanov。2024. Vidur: LLM 推理的大规模模拟框架。arXiv:2405.05465 [cs.LG] https://arxiv.org/abs/2405.05465
- [4] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov 和 Ramachandran Ramjee。2024. 使用 Sarathi-Serve 控制 LLM 推理中的吞吐量与延迟权衡。第 18 届 USENIX 操作系统设计与实现研讨会 (OSDI 24)。USENIX 协会，加利福尼亚州圣克拉拉，117–134。https://www.usenix.org/conference/osdi24/presentation/agrawal
- [5] 米斯特拉尔人工智能。2023. 混合。https://huggingface.co/docs/transformers/main/model_doc/mistral
- [6] 米斯特拉尔人工智能。2023. 混合。https://huggingface.co/docs/transformers/model_doc/mixtral
- [7] 元人工智能。2024. Code LLaMa。https://github.com/Meta-Llama/codellama
- [8] 元人工智能。2024. 骆驼 3.2。https://www.llama.com/
- [9] 一起人工智能。2024 年。Together: 用于构建和运行生成式 AI 的最快云平台。https://www.together.ai/。访问时间：2024 年 3 月 16 日。
- [10] Emmanuel Amaro, Christopher Branner-Augmon, Zhihong Luo, Amy Ousterhout, Marcos K. Aguilera, Aurojit Panda, Sylvia Ratnasamy 和 Scott Shenker。2020. 远记忆能提高工作吞吐量吗？第十五届欧洲计算机系统会议 (希腊伊拉克利翁) 会议记录 (EuroSys '20)。计算机协会，美国纽约州纽约市，第 14 条，第 16 页。https://doi.org/10.1145/3342195.3387522
- [11] 亚马逊网络服务。2024. EC2 Auto Scaling 温池。访问时间：2024 年 10 月 12 日。
- [12] Reza Yazdani Aminabadi, Samyam Rajbhandari, Amar Ahmad Awan, Cheng Li, Du Li, Elton Cheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley 和 Yuxiong He。2022. DeepSpeed-inference: 以前所未有的规模实现变压器模型的高效推理。高性能计算、网络、存储和分析国际会议 (德克萨斯州达拉斯) (SC '22) 论文集。IEEE Press, 第 46 条，15 页。

- [13] 人择。[n. d.]. 提示缓存。https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching。访问时间：2024 年 9 月 11 日。[14] arXiv。2023.ArXiv 总结数据集。https://huggingface.co/datasets/ccdv/arxiv-summarization。访问时间：2024 年 10 月 12 日。[15] 亚马逊网络服务 (AWS)。2024.生成式人工智能用例和资源。https://aws.amazon.com/generative-ai/use-cases/。访问日期：2024 年 3 月 16 日。[16] Microsoft Azure。[n. d.]. Azure 人工智能工作室。访问日期：2024 年 3 月 16 日。网址：https://azure.microsoft.com/en-us/products/ai-studio/。[17] 微软 Azure。2024. Azure LLM 推理数据集 2024。https://github.com/Azure/AzurePublicDataset/blob/master/analysis/AzureLLMInferenceDataset2024.ipynb。访问时间：2025 年 2 月 4 日。[18] 白志豪，张震，朱一波，金鑫。[n. d.]. Orion：用于 ML 应用程序的干扰感知、细粒度 GPU 共享。EuroSys 2024。[19] 白志浩，张震，朱一波，金鑫。2020。PipeSwitch：深度学习应用程序的快速流水线上下文切换。第 14 届 USENIX 操作系统设计与实现研讨会 (OSDI 20)。USENIX 协会，499–514。https://www.usenix.org/conference/osdi20/presentation/bai [20] Yoshua Bengio, Réjean Ducharme 和 Pascal Vincent。2000.神经概率语言模型。《神经信息处理系统进展》，T. Leen, T. Dietterich 和 V. Tresp (编辑)，卷。13。麻省理工学院出版社。https://proceedings.neurips.cc/paper_files/paper/2000/file/728f206c2a01bf572b5940d7d9a8fa4c-Paper.pdf [21] Bob Briscoe, Anna Brunstrom, Andreas Petlund, David Hayes, David Ros, Ing-Jyh Tsang, Stein Gjessing, Gorry Fairhurst, 卡斯滕·格里沃兹和迈克尔·韦尔茨尔。2016。减少互联网延迟：技术及其优点的调查。交流。调查 哎呀。18, 3 (2016 年 7 月)，2149–2196。https://doi.org/10.1109/COMST.2014.2375213 [22] Cerebras 2021。人工智能的未来是晶圆级。https://www.cerebras.net/product-chip/。[23] Sangjin Choi, Taeksoo Kim, Jinwoo Jeong, Rachata Ausavarungnirun, Myeongjae Jeon, Youngjin Kwon 和 Jeongseob Ahn。2022。具有分层统一虚拟内存的多 GPU 系统中的内存收集。2022 年 USENIX 年度技术会议 (USENIX ATC 22)。USENIX 协会，卡尔斯巴德，加利福尼亚州，625–638。https://www.usenix.org/conference/atc22/presentation/choi-sangjin [24] Jacob Devlin, Ming-Wei Chang, Kenton Lee 和 Kristina Toutanova。2019.BERT：用于语言理解的深度双向变压器的预训练。arXiv:1810.04805 [cs.CL] [25] 面向开发人员的 Google AI。[n. d.]. Gemini API 定价。https://ai.google。开发/定价。访问时间：2024 年 9 月 11 日。[26] Amir Gholami, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W. Mahoney 和 Kurt Keutzer。2024.人工智能和记忆墙。IEEE Micro 44, 3 (2024), 33–39。https://doi.org/10.1109/MM.2024.3373763 [27] 谷歌。2024 年。Google 用户放弃统计数据。访问时间：2024 年 10 月 12 日。[28] 谷歌。2024。部分提示。https://huggingface.co/datasets/nateraw/parti-prompts。访问时间：2024 年 5 月 2 日。[29] 顾俊成, Youngmoon Lee, 张一文, 莫沙拉夫·乔杜里和 Kang G. Shin。2017。使用 Infiniswap 进行高效内存分解。第 14 届 USENIX 网络系统设计与实现研讨会 (NSDI 17)。USENIX 协会，波士顿，马萨诸塞州，649–667。https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/gu [30] Arpan Gujarati, Reza Karimi, Safya Alzayat, Weihao, Antoine Kaufmann, Ymir Vigfusson 和 Jonathan Mace。2020 年。像 Clockwork 这样的 DNN 服务：自下而上的性能可预测性。第 14 届 USENIX 操作系统设计与实现研讨会 (OSDI 20)。USENIX 协会，443–462。https://www.usenix.org/conference/osdi20/presentation/gujarati [31] 古罗比。2024.古罗比。https://www.gurobi.com/。访问时间：2024 年 5 月 3 日。[32] 英特尔 Gaudi 人工智能加速器 2021。英特尔 Gaudi 人工智能加速器。https://habana.ai/products/gaudi/。[33] Norman P. Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, Cliff Young, Xiang Zhou, Zongwei Zhou 和 David Patterson。2023. TPU v4：用于机器学习的光学可重构超级计算机，具有嵌入的硬件支持。arXiv:2304.01433 [cs.AR] [34] Felix Kreuk, Gabriel Synnaeve, Adam Polyak, Uriel Singer, Alexandre Défossez, Jade Copet, Devi Parikh, Yaniv Taigman 和 Yossi Adi。2023.AudioGen：文本引导音频生成。arXiv:2209.15352 [cs.SD] [35] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Shen, Lianmin Cheng, Codyhao Yu, Joseph Gonzalez, Hao Zhu 和 Ion Stoica。2023.使用 PagedAttention 进行大型语言模型的高效内存管理。第 29 届操作系统原理研讨会论文集 (<conf-loc>, <city>Koblenz</city>, <country>Germany</country>, </conf-loc>) (SOSP '23)。计算机协会，美国纽约州纽约市，611–626。https://doi.org/10.1145/3600006.3613165 [36] Zhuohan Li, Lianmin Cheng, Yinminzhong, Vincent Liu, Yingheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhu, Joseph E. Gonzalez 和 Ion Stoica。[n. d.]. AlpServe：用于深度学习服务的模型并行统计复用。https://www.usenix.org/conference/osdi23/presentation/li-zhouhan。第 17 届 USENIX 操作系统设计与实现研讨会 (OSDI 23)。[37] Yuhua Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, Junchen Jiang。2024.CacheGen：用于快速大型语言模型服务的 KV 缓存压缩和流。ACM SIGCOMM 2024 会议记录 (澳大利亚新南威尔士州悉尼) (ACM SIGCOMM '24)。计算机协会，美国纽约州纽约市，38–56。https://doi.org/10.1145/3651890.3672274 [38] 微软。2024 年。人工智能正在改变企业。https://azure.microsoft.com/en-us/blog/azure-openai-service-10-ways-generative-ai-is-transforming-businesses/，访问日期：2024 年 3 月 16 日。[39] Tomas Mikolov, Kai Chen, Greg Corrado 和 Jeffrey Dean。2013.向量空间中词表示的有效估计。arXiv:1301.3781 [cs.CL] [40] MosaicML。2023.MPT。https://huggingface.co/docs/transformers/main/model_doc/mpt。[41] 英伟达。2024.NVIDIA Blackwell 架构。https://www.nvidia.com/en-us/data-center/technologies/blackwell-architecture/ 访问时间：2024 年 10 月 12 日。[42] 英伟达。2024. NVLink 和 NVSwitch：最快的 HPC 数据中心平台。https://www.nvidia.com/en-us/data-center/nvlink/。访问时间：2024 年 3 月 16 日。[43] 英伟达公司。2024 年。NVIDIA B200。https://www.nvidia.com/en-us/data-center/dgx-b200/。访问时间：2024 年 4 月 28 日。[44] 英伟达公司。2024.NVIDIA DGX A100 数据表。https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/nvidia-dgx-a100-datasheet.pdf。访问时间：2024 年 4 月 21 日。[45] 英伟达公司。2024 年。NVIDIA H100 张量核心 GPU 数据表。https://resources.nvidia.com/en-us-tensor-core/nvidia-tensor-core-gpu-datasheet。访问时间：2024 年 4 月 21 日。[46] Nvidia DGX 系统 2021。Nvidia DGX 系统。https://www.nvidia.com/en-us/data-center/dgx-systems/。[47] 开放人工智能。2024.OpenAI API。访问日期：2024 年 3 月 16 日。网址：https://openai.com/blog/openai-api。[48] 开放人工智能。2024.OpenAI 批处理 API。https://platform.openai.com/docs/guides/batch。访问时间：2024 年 9 月 11 日。[49] 开放人工智能。2024.OpenAI 客户案例。https://openai.com/customer-stories。访问日期：2024 年 3 月 16 日。

[50] Pratyush Patel, Esha Choukse, Chaojie 张, Aashaka Shah, Íñigo Goiri, Saeed Maleki and Ricardo Bianchini. 2024. Split-wise: 使用相分裂的高效生成 LLM 推理. arXiv:2311.18677 [cs.AR] <https://arxiv.org/abs/2311.18677> [51] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser 和 Björn Ommer. 2022. 具有潜在扩散模型的高分辨率图像合成. arXiv:2112.10752 [cs.CV] [52] 亚马逊网络服务. [n. d.]. 亚马逊 SageMaker. 访问日期: 2024 年 3 月 16 日. 网址: <https://aws.amazon.com/sagemaker/>. [53] 亚马逊网络服务. 2024. Mwtant 案例研究. [urlhttps://aws.amazon.com/solutions/case-studies/mewtant-case-studies/](https://aws.amazon.com/solutions/case-studies/mewtant-case-studies/), 访问于 2024 年 3 月 16 日. [54] 盛颖, 曹诗一, 李大成, 朱邦华, 李卓瀚, 卓丹阳, Joseph E. Gonzalez, Ion Stoica. 2024. 服务大型语言模型的公平性. 第 18 届 USENIX 操作系统设计与实现研讨会 (OSDI 24). USENIX 协会, 加利福尼亚州圣克拉拉, 965–988. <https://www.usenix.org/conference/osdi24/presentation/sheng> [55] Ying Shen, Lianmin Cheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Daniel Y. Fu, Zhiqiang Xie, Beidi Chen, Clark Barrett, Joseph E. Gonzalez, Percy Liang, Christopher Ré, Ion Stoica 和 Ce Zhang. 2023. FlexGen: 使用单个 GPU 进行大型语言模型的高吞吐量生成推理. arXiv:2303.06865 [cs.LG] [56] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper 和 Bryan Catanzaro. 2020. Megatron-LM: 使用模型并行性训练数十亿参数语言模型. arXiv:1909.08053 [cs.CL] <https://arxiv.org/abs/1909.08053> [57] Sudipta Saha Shubha, 沉海英和 Anand Iyer. [n. d.]. USHER: 资源优化机器学习推理的整体干扰避免. <https://www.usenix.org/conference/osdi24/presentation/shubha>. 第 18 届 USENIX 操作系统设计与实现研讨会 (OSDI 24). [58] Arjun Singh, Joon Ong, Amit Agarwal, Glen Anderson, Ashby Armistad, Roy Bannan, Seb Boving, Gaurav Desai, Bob Felderman, Paulie Germano, Anand Kanagala, Jeff Provost, Jason Simmons, Eiichi Tanda, Jim Wampler, Urs Hölzle, Stephen Stuart 和 Amin Vahdat. 2015 年. 木星崛起: 谷歌数据中心网络十年的 Clos 拓扑和集中控制. 在 Sigcomm '15 中. [59] 孙彪, 黄子明, 赵涵宇, 肖文聪, 张歆艺, 李勇, 林伟. 2024. Llumnix: 大型语言模型服务的动态调度. 第 18 届 USENIX 操作系统设计与实现研讨会 (OSDI 24). USENIX 协会, 加利福尼亚州圣克拉拉, 173–191. <https://www.usenix.org/conference/osdi24/presentation/sun-biao> [60] Linux 内核开发人员. [n. d.]. CFS 调度程序. <https://docs.kernel.org/scheduler/sched-design-CFS.html>. 访问时间: 2024 年 3 月 17 日. [61] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser 和 Illia Polosukhin. 2017 年. 您所需要的就是注意力. 《神经信息处理系统进展》, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. V. Ishwanathan 和 R. Garnett (编辑), 卷. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf [62] Abhishek Vijaya Kumar, Arjun Devraj, Darius Bunandar 和 Rachee Singh. 2024 年. 服务器规模光子连接的案例. 第 23 届 ACM 网络热点研讨会论文集 (美国加利福尼亚州尔湾) (HotNets '24). 计算机协会, 美国纽约州纽约市, 290–299. <https://doi.org/10.1145/3696348.3696856>

[63] 大语言模型. 23. 分布式大语言模型. https://docs.vllm.ai/en/latest/serving/distributed_serving.html. 访问时间: 2024 年 10 月 12 日. [64] VLLM. 2024. LLama31: 提高大语言模型效率. <https://blog.vllm.ai/2024/07/23/llama31.html>. 访问日期: 2024 年 9 月 11 日. [65] 吴冰阳, 钟银民, 张自力, 刘胜宇, 刘方跃, 孙远航, 黄刚, 刘轩哲, 金鑫. 2024. 大型语言模型的快速分布式推理服务. arXiv:2305.05920 [cs.LG] <https://arxiv.org/abs/2305.05920> [66] Wencong Shaw, Romil Bhardwaj, Ramachandran Ramjee, Muthian Sivathanu, Nipun Kwatra, Zhenhua Han, Pratyush Patel, Xuan Peng, Hanyu Zhu, Quanlu Zhang, Fan Yang 和 Lidong Zhou. 2018. Gandiva: 深度学习的内省集群调度. 第 13 届 USENIX 操作系统设计与实现研讨会 (OSDI 18). USENIX 协会, 卡尔斯巴德, 加利福尼亚州, 595–610. <https://www.usenix.org/conference/osdi18/presentation/xiao> [67] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim 和 Byung-Gon Chun. 2022. Orca: 基于 Transformer 的生成模型的分布式服务系统. 第 16 届 USENIX 操作系统设计与实现研讨会 (OSDI 22). USENIX 协会, 卡尔斯巴德, 加利福尼亚州, 521–538. <https://www.usenix.org/conference/osdi22/presentation/you> [68] Peifeng Yu 和 Mosharaf Chowdhury. 2019. Salus: 深度学习应用的细粒度 GPU 共享原语. arXiv:1902.04610 [cs.DC] [69] 赵旋雷, 贾斌, 周浩天, 刘子明, 程胜干和尤阳. 2024. HeteGen: 资源受限设备上大型语言模型的异构并行推理. <https://arxiv.org/abs/2403.01164>. 访问日期: 2024 年 9 月 11 日. [70] Yinminzhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: 分解预填充和解码以实现 Goodput 优化的大型语言模型服务. 第 18 届 USENIX 操作系统设计与实现研讨会 (OSDI 24). USENIX 协会, 加利福尼亚州圣克拉拉, 193–210. <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>

神器附录

A.1 摘要

我们在 <https://github.com/aquaml> 上发布了 Aqua 的源代码. com/aquaml. Aqua 需要一个多 GPU 服务器, 其中至少有 2 个通过 Nvlink 互连的 NVIDIA GPU. Aqua 的文档可在 <http://aquaml.github.io/> 上获取.

A.2 工件检查表 (元信息)

以下是工件中可用的存储库的列表.

- Aqua-lib 包含 Aqua 张量.
- 使用 Aqua 的 CFS 算法的 vLLM
- Aqua-放置者.
- 带有集成 Aqua-lib 的 PyTorch 脚本用于执行稳定扩散.
- FlexGen 带有集成的 Aqua-lib.
- Audiogen 集成了 Aqua-lib.
- 是否公开? : 是